

1ST SCARLINE WORKSHOP

On Operationalization of Driving Simulator Studies for Accessible In-Vehicle HMI Prototyping and Evaluation

Welcome to the 1st SCARline Workshop at AutoUI 2026! 😊

Today, we'll introduce SCARline and explore how it can support the rapid prototyping and evaluation of in-vehicle HMIs in driving simulator studies.

We've prepared this handout to help you **follow along during the workshop** and serve as a **reference for using SCARline afterward**.

WHAT WE'LL DO TODAY

Session 1: Introduction

We will introduce you to SCARline and you will hear two interesting presentations.

Session 2: Walkthrough

We will show you the platform and answer any questions you may have.

Session 3: Widget Creation

We will explain how widgets can be created. As groups, you then get to design your own widgets.

Coffee break

Session 4: Live Deployment

You will have the opportunity to explore SCARline and deploy the widgets you created.

Session 5: Feedback

We'd love to hear your feedback in the form of a discussion and a few questions.

HOW THIS HANDOUT IS STRUCTURED

Part A: What is SCARline?

A short introduction to SCARline, its architecture, and how its components work together.

Part B: Your Task Today

An exercise where you get to build your own widget and then run it with SCARline.

If you have questions during the workshop, just ask! We're happy to help.

DIGITAL VERSION



tugcanonbas.github.io/SCARline/

PART A: WHAT IS SCARLINE?

SCARline sits on top of a driving simulator and handles the technical complexity underneath, so you can focus on your research question and study design rather than fighting with the tools. You don't need to know how the simulator works internally — just what you're testing and how.

That doesn't mean SCARline takes the creative work away from you. You're still the one designing the interaction, the platform just removes the plumbing between your idea and a running study.

SCARLINE AT A GLANCE:

1. A plug-and-play UX platform for driving simulation research.
2. One system to build, configure, run, and review a study from start to finish.
3. Built for researchers and designers, from no-code builds to full custom code.
4. Open source and available at github.com/tugcanonbas/SCARline.



Scan me for the Github

1. SCARLINE ARCHITECTURE

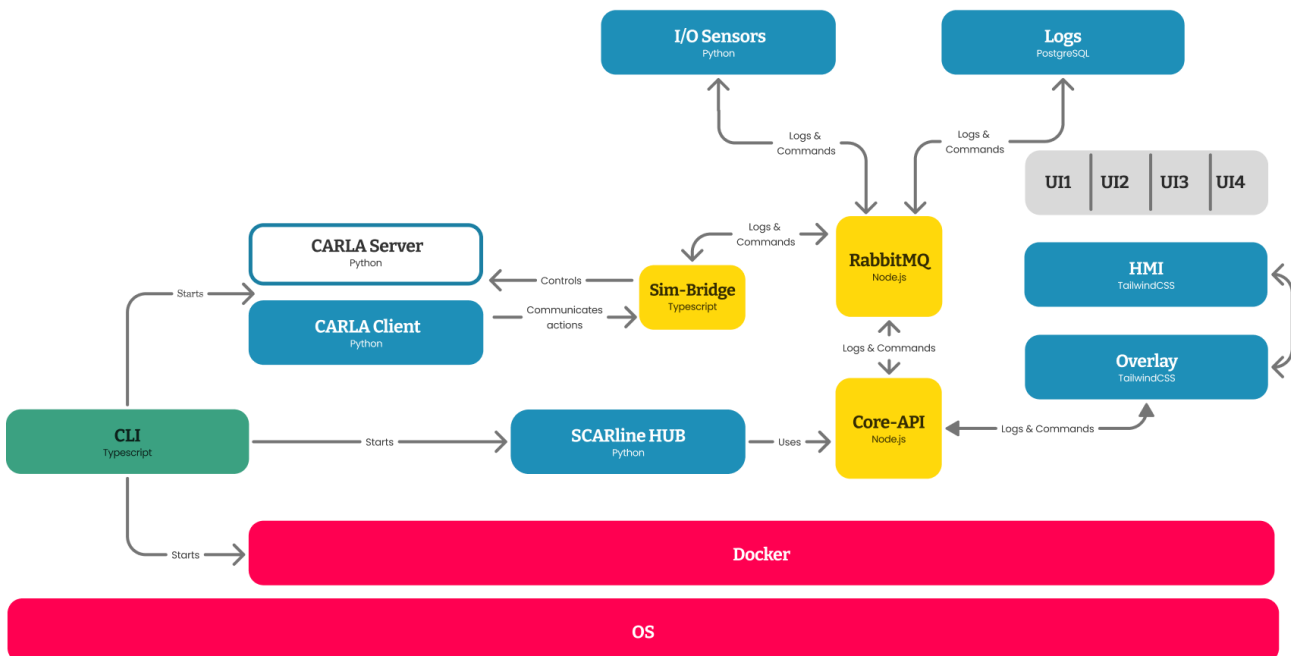


Figure 1: SCARline's full system architecture — This includes the CLI, SCARline Hub, CARLA simulator, Sim-Bridge, RabbitMQ, Core API, I/O sensors, PostgreSQL logging, and the HMI/overlay layer. Docker provides the underlying containerized runtime environment.

INFORMATION FLOW

When something changes in the simulation (e.g., the vehicle's speed reaches 80 km/h) CARLA (the driving simulator itself) doesn't talk to your widget directly. It first passes that value to the Sim-Bridge, whose job is to translate raw simulator state into messages the rest of the system understands. The Sim-Bridge publishes that message to RabbitMQ, a message queue that sits between the simulator side and the widget side of the pipeline. The message doesn't go anywhere on its own from there. It waits in the queue until something is ready to pick it up.

That something is the Core-API, which consumes the message from RabbitMQ, then processes and packages it into the shape your widget actually needs (e.g., turning "speed = 80km/h" into whatever data field your widget is listening for).

Once packaged, the Core-API pushes that data to the Overlay over a websocket connection. This is a live, always-open channel rather than a one-off request. The Overlay is what updates the widget on screen in response.

Short version: simulator state flows out through Sim-Bridge → queue → Core-API → live push → your widget on screen, with RabbitMQ acting as a buffer in the middle rather than a direct handoff.

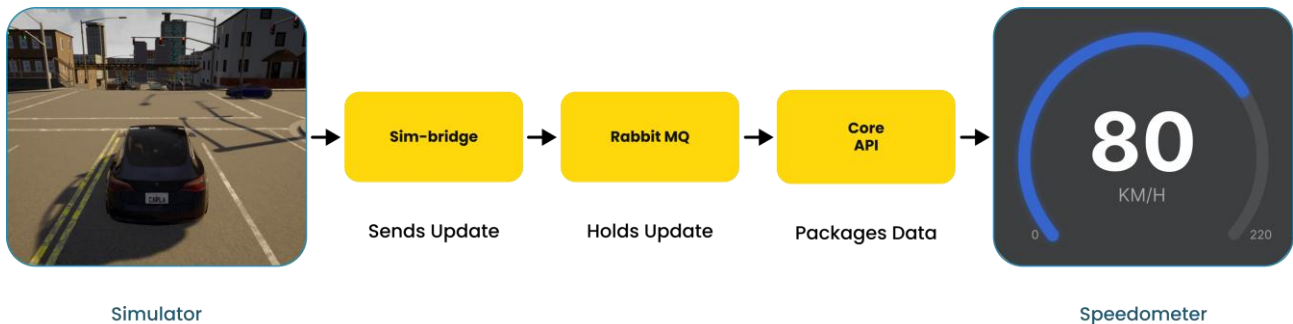


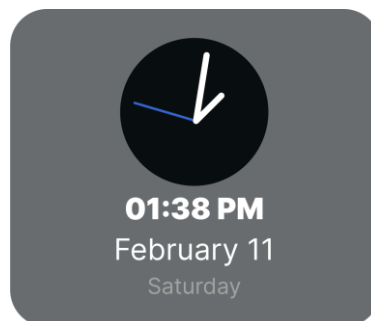
Figure 2: How a single data point travels from the simulator to your widget on screen.

2. WIDGETS

A widget is the piece of interface a driver or passenger actually sees and interacts with inside the car. This could for example be a heart-rate readout on the dashboard, a warning icon that appears when you're following too closely, a passenger-side screen showing trip stats. Widgets are connected to the simulation, so it can change based on what's happening in the drive, e.g., speed, distance to an obstacle, elapsed time, or whatever state you choose to hook it to. Every widget has, at minimum, a resting state (what it looks like normally) and a triggered state (what changes when the condition you define is met).



a) Speedometer widget



b) Clock, including date

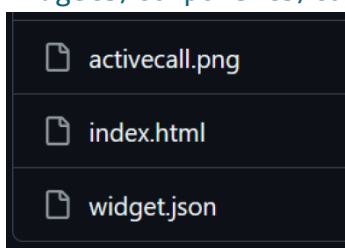


c) EKG displaying heart rate and blood pressure

Figure 3: Example widgets present in SCARline.

WIDGET STRUCTURE

Every widget lives in its own folder inside `widgets/components/` (e.g., `widgets/components/calendar/`) and is made up of three files:



- A preview image (e.g., `activecall.png`) — a snapshot of the widget shown in the SCARline interface editor, so it's recognizable without loading the live widget.
- `index.html` — the widget's visual structure, styled with Tailwind CSS classes.
- `widget.json` — the configuration file: it defines the widget's dimensions and, most importantly, the data bindings it expects to receive.

HOW WIDGETS RECEIVE DATA

Data binding allows your widget to receive live data from the simulator. This is handled through the `data-bind` attribute in your html file by connecting it to the matching key in `widget.json`. For example: `<span data-bind="calendar.date_label">11</span>`

In your json file this would then be:

```
1  "bindings": {  
2    "calendar.date_label": {  
3      "type": "string",  
4      "description": "Current day-of-month label.",  
5      "default": "11"  
6    },  
7  }
```

SENDING ACTIONS OUT

Buttons or other interactive elements can change what a widget shows through `data-action`. When clicked, the shared runtime sends that action to the SCARline system:

```
<button type="button" data-action="calendar.add_event">+</button>
```

Most widgets are read-only, so just displaying data.

If a widget needs something more custom, it can also send events directly through the SCARline API, though that's rarely needed for a first widget. You would do this with a custom script:

```
1  <script>  
2    window.SCARline.send("widget.interaction", {  
3      widgetId: "appointments",  
4      action: "add_event"  
5    });  
6  </script>
```

The binding and the action handling runs through a shared runtime script (`<script src="../../widget-runtime.js"></script>`) loaded by every widget, so you never have to write that plumbing yourself.

BUILD OR RECREATE A WIDGET

Create a new folder inside `components/`, draft your `index.html` with Tailwind classes, define what data it expects in `widget.json`, and run `npm run dev` inside the widgets folder to preview and tweak it live in your browser — before it's picked up by the rest of the system.

PART B: YOUR TASK TODAY (BUILD YOUR OWN WIDGET)

THE SITUATION

You have just joined a research group that uses SCARline for driving simulation studies. Your first project is to **understand how drivers interact with in-cabin health monitoring displays** while on the road.

Health monitoring displays are meant to keep a driver informed without pulling their attention off the road.

YOUR TASK

Your team wants to know: **How can a widget communicate what it needs to, at the moment it matters, without becoming a distraction?**

The specifics (what's being monitored, what the alert looks like, when it triggers) are yours to decide. *Treat it as your first design decision, not a constraint to work around.*

In this part of the workshop, your team will:

1. Define your widget idea (you can use the template below).
2. Build it — either with AI-assisted coding, coding yourself, sketching it out and having a teammate implement it.
3. Validate and upload your widget to SCARline.

You don't need prior programming or design experience — organizers will be on hand throughout. Aim to have a validated, uploaded widget by the end of this part; a working widget beats a perfect one.



Once you've finished the widget, you can zip the folder and upload it:

drive.google.com/drive/folders/1O3JR_xyeeB5r3YL0revaqSaoLMS7IbKE?usp=sharing

1. WAYS TO BUILD

You don't need one person who can code. Split the work however fits your team:

- a) Code it yourselves: write the widget's UI directly.
- b) Sketch it, teammate codes it: one person sketches the layout and states, another turns it into code.
- c) Describe it to AI: tell an AI assistant what the widget should look like and do, in words, and let it generate the code.
- d) Show AI a sketch: draw or mock up the widget, hand the image to AI, and let it generate the code from that.

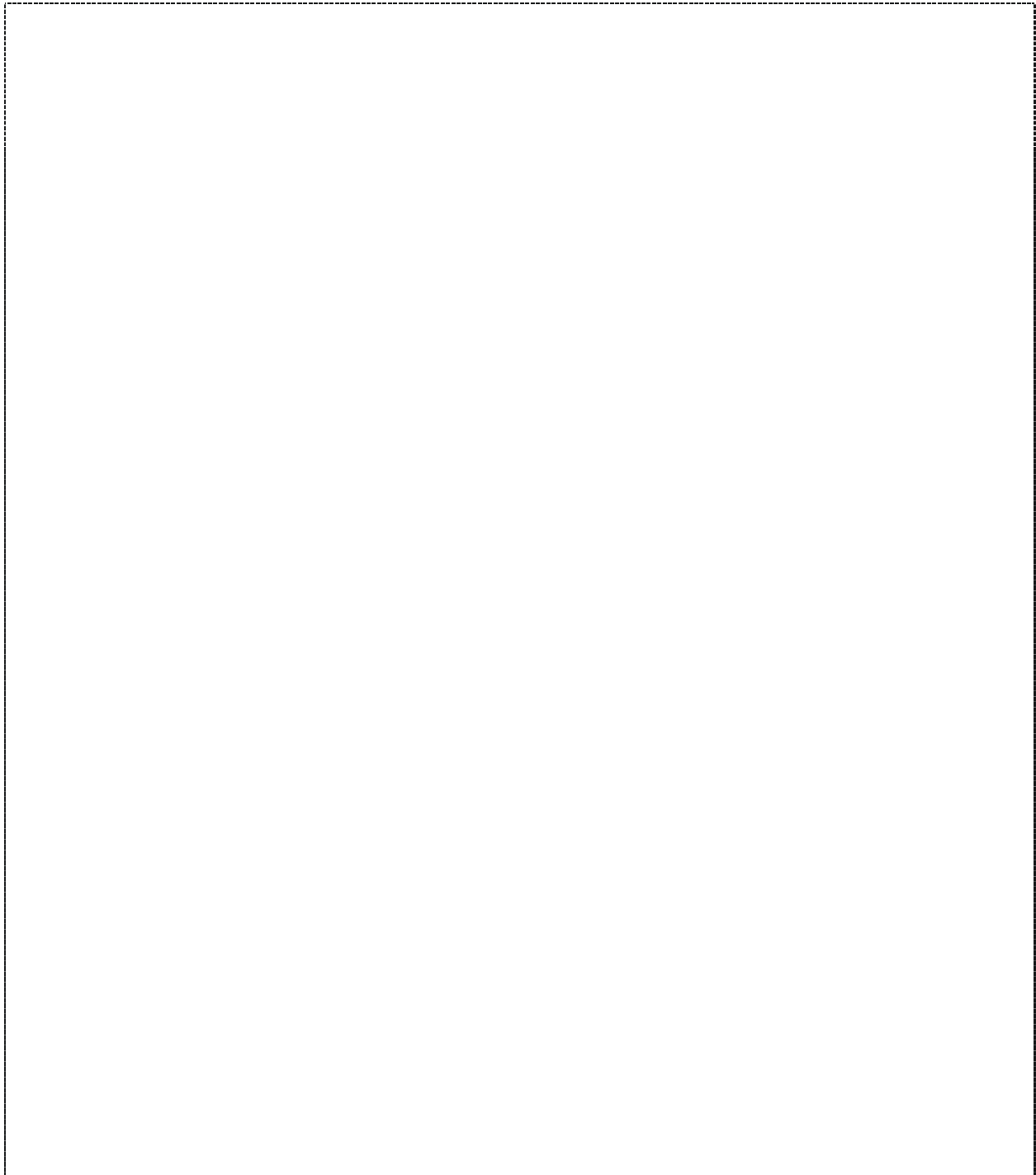
Whatever your team produces, you'll validate and upload it the same way.

2. DEFINE YOUR WIDGET – IDEA TEMPLATE

Before you open any tool, take five minutes to answer these questions. A clear idea is the fastest path to a working widget.

- What does your widget do? What information/interaction does it give the driver or passenger?
- When does it appear? What vehicle or simulation state should trigger it or make it change (e.g. speed, an obstacle, distance to a stop)?
- Who is it for? Is it meant for the driver, a passenger, or both and in what kind of scenario?
- What does it look like at rest, and what changes when it's triggered?
- What data does it need as input, and what should it record for later analysis?

Sketch it out:



3. ADDITIONAL INFORMATION

REPOSITORY

This contains all files and a readme for more detailed and technical information.

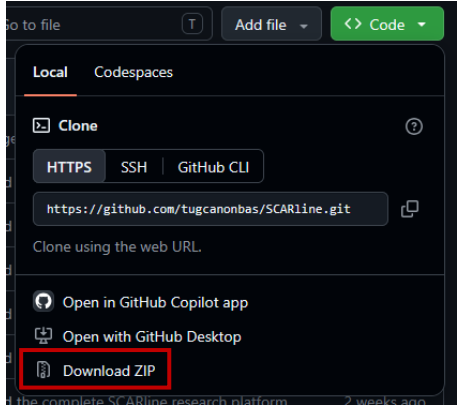


 github.com/tugcanonbas/SCARline

SCARline Requirements (as of 20.09.2026):

- Node.js 22 or newer
- npm 10 or newer
- Docker
- Git (optional)

COMMANDS

Task	Command
Installing SCARline	Step 1: OPTION A: WITH GIT Open Git Bash and enter the following command: <pre>git clone https://github.com/tugcanonbas/SCARline.git</pre> OPTION B: WITHOUT GIT  1) Navigate to the Github repository 2) Click on the "Code" button on the top 3) Click "Download ZIP" 4) Unzip the folder inside your files. Step 2: Inside your console enter the following commands one by one: <pre>cd SCARline-main npm install scarline setup</pre>
Validating installation	<pre>scarline status</pre>
Running SCARline	<pre>scarline start</pre>
Exiting SCARline	<pre>scarline stop</pre>
Building your widget	Create a folder inside <code>widgets/components/</code> and name it accordingly to your widget. Ensure that this folder contains a .png, .html, and .json file.

4. TIPS

- Save your work often so you don't lose progress.
- Time is limited: if you get stuck for more than a few minutes on setup or tooling, flag a facilitator rather than troubleshooting alone.
- You'll be asked to fill out a short questionnaire at the end — this is separate from this handout and helps us improve future SCARline workshops and the platform.
- This handout is yours to keep and annotate — write directly on it as you work through the template.

Questions during the workshop? Just ask — that's what we're here for.